

Sistema Web de Avaliacao de Disciplinas

Documentacao Tecnica e Fluxos de Funcionamento

Universidade Estadual de Campinas
Faculdade de Tecnologia

Autor: Victor Valentim Bergamasco
Orientador: Prof. Dr. Vitor Rafael Coluci

Maio / 2026

Sumario

1. Visao Geral do Sistema
2. Arquitetura e Stack Tecnologica
3. Estrutura de Pastas do Projeto
4. Backend Django - Camada de Servidor
 - 4.1 core - Configuracao e roteamento global
 - 4.2 apps/users - Autenticacao e usuarios
 - 4.3 apps/courses - Disciplinas e QR Code
 - 4.4 apps/evaluations - Perguntas, respostas e formulario publico
 - 4.5 apps/dashboard - Visao geral e estatisticas
 - 4.6 apps/reports - Geracao do relatorio PDF
5. Frontend React - Camada de Cliente
 - 5.1 services/api.js - Cliente HTTP
 - 5.2 components/ - Layout e Sidebar
 - 5.3 pages/ - Telas do sistema
 - 5.4 App.jsx - Rotas e protecao
6. Modelo de Dados (Banco)
7. Fluxos de Funcionamento
 - 7.1 Cadastro de professor pelo admin
 - 7.2 Primeiro login do professor (troca obrigatoria de senha)
 - 7.3 Cadastro de disciplina e geracao de QR Code
 - 7.4 Aluno responde o formulario via QR
 - 7.5 Professor consulta dashboard
 - 7.6 Geracao do relatorio PDF
8. API REST - Endpoints Disponiveis
9. Seguranca, Anonimato e Privacidade
10. Proximos Passos

1. Visao Geral do Sistema

Este sistema web automatiza o processo de avaliacao de disciplinas da Faculdade de Tecnologia (FT) da UNICAMP. Hoje, os professores compartilham QR Codes com os alunos e recebem os dados brutos das respostas, precisando organizar manualmente os resultados. A proposta deste TCC e digitalizar e automatizar todo o ciclo: cadastro de disciplinas, geracao de QR Codes, coleta anonima de respostas, calculo estatistico das medias e geracao automatica de relatorios em PDF para os professores.

O sistema possui dois tipos de usuarios autenticados (administrador e professor) e um terceiro participante nao autenticado: o aluno, que responde anonimamente o formulario publico ao escanear o QR Code. Cada papel ve apenas o que lhe e pertinente, e nenhuma resposta carrega identificacao do respondente, preservando o anonimato exigido pela politica de avaliacao institucional.

Objetivos principais

- bullet Cadastrar professores, disciplinas e gerar QR Codes unicos por disciplina.
- bullet Disponibilizar formulario publico, acessivel via QR Code, sem login.
- bullet Armazenar respostas de forma anonima em banco relacional.
- bullet Calcular medias, desvios padrao e tamanho amostral para cada disciplina.
- bullet Gerar relatorio consolidado em PDF com graficos, comentarios e estatisticas.
- bullet Restringir acesso: cada professor so visualiza suas proprias disciplinas.

2. Arquitetura e Stack Tecnologica

O sistema segue o padrao cliente-servidor com tres camadas independentes que se comunicam exclusivamente via API REST sobre HTTP/JSON:

Camada	Tecnologia
Frontend (cliente)	React 18 + Vite + React Router + Axios + Recharts
Backend (servidor)	Django 5.0 + Django REST Framework + SimpleJWT
Banco de dados	PostgreSQL 14+
Geracao de PDF	ReportLab + Matplotlib (no backend)
QR Code	Biblioteca qrcode (Pillow) no backend
Autenticacao	JWT (JSON Web Tokens) via simplejwt

Por que essa pilha

- React:** SPA com interface reativa, componentes reutilizaveis, ecossistema rico (Recharts para graficos, React Router para navegacao).
- Django REST Framework:** produtividade alta, ORM robusto, permissoes declarativas, serializers expressivos e autenticacao JWT bem integrada.
- PostgreSQL:** banco relacional confiavel, com integridade referencial garantida por ForeignKey e operacoes ACID.
- ReportLab + Matplotlib:** geracao do PDF no servidor garante consistencia visual e permite calculo estatistico antes do render.

Diagrama de comunicacao

O fluxo de uma requisicao tipica e:

```
Navegador (React)          Servidor (Django)          Banco (Postgres)
|                           |                           |
|--- POST /api/auth/login/ ---->|                           |
| (institutional_email, senha) |--- SELECT user ----->|
|                               |<-- linha do usuario ----- |
|<--- access token (JWT) -----|                           |
|                               |                           |
|--- GET /api/courses/ ----->|                           |
|   Authorization: Bearer ...   |--- SELECT courses ----->|
|<--- JSON com disciplinas -----|<-- linhas das disciplinas --- |
```

3. Estrutura de Pastas do Projeto

O projeto é organizado em duas pastas principais (backend e frontend), cada uma com seu ciclo de vida próprio (rodam em portas diferentes em desenvolvimento, e seriam deployadas separadamente em produção).

```
TCC/
|-- backend_tcc/                               Servidor Django (porta 8000)
|   |-- core/                                  Configuracao do projeto Django
|   |   |-- settings.py                       Banco, JWT, CORS, apps instaladas
|   |   |-- urls.py                           Roteamento global (/api/...)
|   |   |-- wsgi.py                           Entrada para servidor WSGI
|   |   |-- asgi.py                           Entrada para ASGI (caso WebSocket)
|   |-- apps/
|   |   |-- users/                            Usuarios, JWT, primeiro login
|   |   |-- courses/                          Disciplinas e QR Code
|   |   |-- evaluations/                      Perguntas e respostas
|   |   |-- dashboard/                       Estatisticas
|   |   |-- reports/                         Geracao de PDF
|   |-- media/                                Arquivos gerados (QR Codes, PDFs)
|   |   |-- qrcodes/
|   |   |-- reports/
|   |-- manage.py                             CLI do Django
|   |-- requirements.txt                       Dependencias Python
|   |-- .env                                  Variaveis sensiveis (nao versionado)
|
|-- frontend_tcc/                              Aplicacao React (porta 5173)
|   |-- src/
|   |   |-- main.jsx                          Entry point do React
|   |   |-- App.jsx                           Definicao de rotas
|   |   |-- pages/                           Telas (Login, Admin, Professor, ...)
|   |   |-- components/                      Componentes reutilizaveis
|   |   |-- services/api.js                   Cliente Axios + JWT interceptor
|   |   |-- styles/                           CSS global
|   |-- public/                               Estaticos servidos diretamente
|   |-- package.json                          Dependencias JS
|   |-- vite.config.js                        Build / dev server
```

4. Backend Django - Camada de Servidor

4.1 core/ - Configuracao e roteamento global

A pasta **core/** contem as configuracoes globais do projeto Django, compartilhadas por todas as apps.

core/settings.py

Pontos-chave da configuracao:

- bullet** **AUTH_USER_MODEL = "users.User"**: usa o usuario customizado da app users (autenticado por e-mail institucional, nao por username).
- bullet** **DATABASES**: PostgreSQL com credenciais lidas do arquivo .env (via python-dotenv), permitindo trocar de banco entre dev e producao sem mexer no codigo.
- bullet** **INSTALLED_APPS**: Django padrao + corsheaders + rest_framework + simplejwt + as cinco apps proprias (users, courses, evaluations, dashboard, reports).
- bullet** **REST_FRAMEWORK**: autenticacao default via JWT e permissao default IsAuthenticated, ou seja, qualquer endpoint exige token Bearer salvo explicito AllowAny.
- bullet** **SIMPLE_JWT**: token de acesso vale 4 horas, refresh dura 7 dias.
- bullet** **CORS_ALLOW_ALL_ORIGINS = True**: aceita requisicoes de qualquer origem (modo desenvolvimento). Em producao seria restrito ao dominio do frontend.
- bullet** **MEDIA_ROOT / MEDIA_URL**: define onde os QR Codes e PDFs gerados ficam armazenados e servidos.

core/urls.py

Roteamento de primeiro nivel. Encadeia os *urls.py* de cada app sob o prefixo */api/*, alem de expor os endpoints de autenticacao JWT.

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('api/auth/login/', TokenObtainPairView.as_view()),  
    path('api/auth/refresh/', TokenRefreshView.as_view()),  
    path('api/users/', include('apps.users.urls')),  
    path('api/courses/', include('apps.courses.urls')),  
    path('api/evaluations/', include('apps.evaluations.urls')),  
    path('api/dashboard/', include('apps.dashboard.urls')),  
    path('api/reports/', include('apps.reports.urls')),  
]
```

4.2 apps/users - Autenticacao e usuarios

Esta e a app mais sensivel do sistema. Define o modelo de usuario customizado, regras de permissao por papel e o fluxo de primeiro login com troca obrigatoria de senha.

Modelo User (models.py)

Substitui o usuario padrao do Django. Em vez de *username*, autentica por *institutional_email*. Possui um campo *role* que pode ser "admin" ou "professor", e o flag *must_change_password* que aciona o fluxo de troca obrigatoria no primeiro acesso.

```
class User(AbstractUser):
    institutional_email = EmailField(unique=True)
    role = CharField(choices=[('admin', 'Administrador'),
                              ('professor', 'Professor')])
    must_change_password = BooleanField(default=False)
    USERNAME_FIELD = 'institutional_email'
    REQUIRED_FIELDS = ['first_name', 'last_name']
```

Permissoes (permissions.py)

Duas classes de permissao customizadas sao usadas em todo o sistema:

- IsAdminRole**: somente usuarios com role='admin'. Aplicada em criacao/edicao de professores, criacao de disciplinas, dashboard global.
- IsProfessorOrAdmin**: aceita ambos. Usada em listagem de disciplinas, geracao de relatorio (com filtro adicional para professor so ver as proprias).

Views principais (views.py)

- MeView**: GET /api/users/me/ retorna os dados do usuario logado (inclusive *must_change_password*). E a primeira chamada apos o login.
- ChangePasswordView**: POST /api/users/me/change-password/ permite ao usuario logado definir nova senha. No fluxo de primeiro login a senha atual nao e exigida; nas demais trocas e obrigatoria.
- ProfessorListCreateView**: GET lista os professores e POST cria um novo. Acessivel apenas por admin.
- ProfessorDetailView**: GET/PATCH/DELETE para gerenciar um professor especifico.

Logica do primeiro login

Quando o admin cadastra um professor (via *UserCreateSerializer*), o flag *must_change_password* e forçado para True. Quando esse professor faz o primeiro login:

- O frontend chama /api/users/me/ e recebe *must_change_password=True*.
- Em vez de ir para o dashboard, e redirecionado para /trocar-senha.
- Ao enviar a nova senha, o *ChangePasswordView* aceita sem exigir senha atual e zera o flag.
- Apos isso, qualquer troca futura exige a senha atual.
- Se o admin posteriormente resetar a senha de um professor, o flag volta para True automaticamente, forçando troca novamente.

4.3 apps/courses - Disciplinas e QR Code

Gerencia o ciclo de vida das disciplinas. Cada disciplina tem um professor responsavel, um codigo unico, um numero de matriculados (usado no calculo amostral do relatorio) e um link de formulario publico que gera automaticamente um QR Code.

Modelo Course (models.py)

```
class Course(models.Model):
    name = CharField(max_length=200)
    code = CharField(max_length=50, unique=True)
    professor = ForeignKey(User, limit_choices_to={'role':'professor'})
    enrollment_count = PositiveIntegerField(default=0)
    google_form_link = URLField()
    qr_code_image = ImageField(upload_to='qrcores/', blank=True)
    is_active = BooleanField(default=True)
    created_at = DateTimeField(auto_now_add=True)
```

Comportamento automatico no save()

O metodo `save()` da classe `Course` foi sobrescrito para executar duas acoes automaticas:

- Gerar QR Code:** sempre que o link do formulario muda (ou na criacao), a funcao `generate_qr_code()` usa a biblioteca `qrcode` para criar um PNG e salva no `ImageField` `qr_code_image`.
- Criar perguntas padrao:** na primeira criacao da disciplina (detectado por `self._state.adding`), chama `sync_standard_questions(self)` da app `evaluations`, garantindo que toda disciplina nova ja nasca com as 24 perguntas oficiais.

Views e permissoes

- GET /api/courses/:** lista disciplinas. Admin ve todas; professor ve apenas as proprias (filter por `professor=user`).
- POST /api/courses/:** cria disciplina. Apenas admin (`get_permissions` sobrescrito).
- GET/PATCH/DELETE /api/courses/<id>/:** detalhe; PATCH/DELETE apenas admin.

Comando sincronizar_perguntas

Comando Django para popular as perguntas padrao em disciplinas antigas (criadas antes da padronizacao). Localizado em `apps/courses/management/commands/sincronizar_perguntas.py`.

```
# Adiciona perguntas faltantes preservando as existentes
python manage.py sincronizar_perguntas

# Atualiza tambem o texto das perguntas existentes
python manage.py sincronizar_perguntas --sobrescrever

# Roda apenas para uma disciplina
python manage.py sincronizar_perguntas --disciplina EST001
```

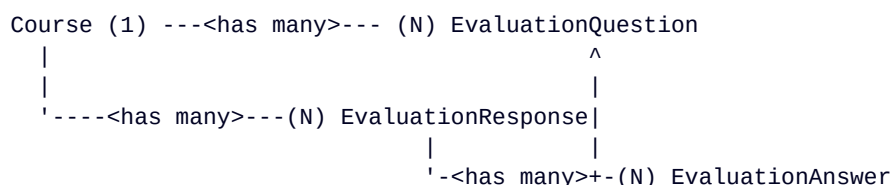
4.4 apps/evaluations - Perguntas, respostas e formulario publico

Coracao do sistema. Define a estrutura das perguntas, recebe as respostas anonimas dos alunos e disponibiliza um endpoint publico (sem login) para a submissao via QR Code.

Modelos (models.py)

- EvaluationQuestion:** pertence a uma Course, possui texto, tipo (scale ou text) e order (numero da pergunta no formulario, 1 a 24).
- EvaluationResponse:** representa uma submissao completa do formulario por um aluno. Nao tem FK para User - garante anonimato. Tem submitted_at e external_response_id (opcional, para casos onde o formulario externo tem ID proprio).
- EvaluationAnswer:** cada resposta individual a uma pergunta. Tem scale_value (0-5) OU text_value (comentario aberto). Ligada a uma EvaluationResponse e a uma EvaluationQuestion.

Diagrama logico das tabelas



EvaluationResponse NAO tem ForeignKey para User -> anonimato

Lista oficial de perguntas (standard_questions.py)

Modulo novo criado para centralizar as 24 perguntas oficiais da FT-UNICAMP. E a unica fonte da verdade do sistema. Estrutura:

- bullet **A) Infraestrutura e Suporte as aulas:** perguntas 1 a 5 (escala).
- bullet **B) Participacao do estudante:** perguntas 6 a 12 (escala).
- bullet **C) Atuacao Docente:** perguntas 13 a 22 (escala).
- bullet **D) Questoes abertas:** perguntas 23 e 24 (texto livre).

A funcao `sync_standard_questions(course, overwrite_text=False)` garante idempotencia: pode ser chamada varias vezes que so cria as perguntas faltantes, sem duplicar. Com `overwrite_text=True` ela atualiza tambem o texto de perguntas ja existentes.

Views (views.py)

- QuestionListCreateView:** GET publico (formulario do aluno) filtrado por course_id; POST restrito a admin.
- PublicEvaluationSubmitView:** POST /api/evaluations/submit/ publico, sem autenticao. Recebe a submissao completa do aluno (course + lista de answers).
- CourseReportView:** GET /api/evaluations/report/<id>/ retorna os dados ja agregados (media por pergunta, distribuicao, comentarios). Usado pela tela Relatorios e tambem pelo gerador de PDF.

4.5 apps/dashboard - Visao geral e estatisticas

App pequena, focada em fornecer estatisticas resumidas para as telas iniciais do admin e do professor.

DashboardOverview

Endpoint GET /api/dashboard/overview/ acessivel apenas pelo admin (IsAdminRole). Retorna tres totais:

```
{
  "total_professors": User.objects.filter(role='professor').count(),
  "total_courses": Course.objects.count(),
  "total_responses": EvaluationResponse.objects.count()
}
```

Importante: total_responses conta submissoes **unicas** do formulario (EvaluationResponse), nao a soma de todas as respostas individuais (EvaluationAnswer). Isso evita inflar o numero exibido (1 aluno respondendo 24 perguntas conta como 1, nao 24).

CourseDashboard

Endpoint GET /api/dashboard/course/<id>/ retorna a media de cada pergunta de escala da disciplina. Usado pelo grafico de barras na tela Relatorios. Caracteristicas:

- bullet Permissao IsProfessorOrAdmin.
- bullet Usa get_object_or_404 para responder 404 limpo se a disciplina nao existir.
- bullet Restringe acesso: professor so consegue ver o dashboard das proprias disciplinas. Tentar acessar outra retorna 403.
- bullet Calcula a media via aggregate(Avg('scale_value')).

4.6 apps/reports - Geracao do relatorio PDF

App mais sofisticada do backend. Gera dinamicamente um PDF formal com graficos, estatisticas e comentarios para qualquer disciplina, usando matplotlib (para os graficos) e reportlab (para a montagem do PDF).

services.py - logica principal

O arquivo services.py contem todo o pipeline. Funcoes-chave:

- tamanho_amostrado(matriculados, tipo):** calcula o n minimo de respondentes usando formula classica com $z=1,96$ (95% de confianca), $p=0,5$ e margem de erro $e=0,15$ para relatorios por professor.
- get_scale_values / get_text_values:** extrai respostas de uma pergunta como listas planas.
- calculate_mean_sd(values):** calcula media e desvio padrao ignorando respostas "Nao sei avaliar" (valor 0).
- create_histogram(values, ordem, dir):** usa matplotlib para gerar um PNG do histograma com porcentagens em cima de cada barra e legenda DT/DP/Neutro/CP/CT.
- header_block(...):** monta o cabecalho institucional (UNICAMP + FT), titulo do relatorio e tabela com metadados da disciplina.
- generate_course_pdf_report(course):** orquestrador. Monta o PDF completo, separando perguntas por secao (A, B, C, D), inserindo histograma para cada pergunta de escala e listando comentarios individuais para perguntas abertas.

Estrutura visual do relatorio

Cada PDF gerado tem o seguinte layout:

- Pagina 1 - cabecalho:** titulo UNICAMP/FT, tabela com disciplina, codigo, professor, respondentes/matriculados, data de geracao, alerta se amostra esta abaixo do minimo, e nota sobre o calculo amostral.
- Paginas das perguntas de escala:** agrupadas pelas secoes A, B e C. Cada pergunta ocupa uma pagina com seu enunciado, histograma com porcentagens, media e desvio padrao, e (na primeira pergunta) a legenda das siglas DT/DP/Neutro/CP/CT.
- Paginas das perguntas abertas (secao D):** cada uma lista todos os comentarios individuais recebidos. Se nao houve comentarios, imprime "Nao houve nenhum comentario".
- Rodape em todas as paginas:** linha azul + texto "Sistema Web de Avaliacao de Disciplinas" + numero da pagina.

Endpoint

```
GET /api/reports/course/<course_id>/pdf/  
Authorization: Bearer <token>
```

-> retorna o arquivo PDF como download.
Admin acessa qualquer disciplina; professor apenas as proprias.

5. Frontend React - Camada de Cliente

Aplicacao construida com React + Vite, organizada em pages (telas inteiras), components (pedacos reutilizaveis) e services (camada de comunicacao com o backend).

5.1 services/api.js - Cliente HTTP

Encapsula o Axios com baseURL apontando para o backend e um *request interceptor* que injeta automaticamente o token JWT no header Authorization de toda requisicao. Tambem expoe getMediaUrl para construir URLs absolutas de imagens (QR Codes) servidas pelo Django.

```
const api = axios.create({ baseURL: 'http://127.0.0.1:8000/api/' });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

5.2 components/ - Layout e Sidebar

- Layout.jsx**: envolve as telas autenticadas. Renderiza a Sidebar a esquerda e o conteudo a direita. Resolve automaticamente o papel (Admin/Professor) lendo do localStorage se nao receber via prop - foi exatamente o fix do bug onde Relatorios mostrava "Admin" mesmo logado como professor.
- Sidebar.jsx**: menu lateral que muda os itens conforme o papel. Admin ve [Inicio, Professores, Disciplinas, Relatorios]; Professor ve [Disciplinas, Relatorios]. Botao "Sair" limpa o localStorage e redireciona para /.
- StatCard.jsx**: card simples para exibir um numero grande com legenda, usado nos cards do dashboard do admin.

5.3 pages/ - Telas do sistema

Login.jsx

Tela publica. POSTa institutional_email + password no /api/auth/login/, salva o token JWT em localStorage e em seguida chama /api/users/me/ para obter o papel. Redireciona:

- Se `must_change_password=true`: vai para /trocar-senha.
- Se professor: vai para /professor (ProfessorDashboard).
- Se admin: vai para /admin (AdminDashboard).

SetPassword.jsx

Tela exclusiva do fluxo de primeiro login (ou troca posterior). Pede a senha atual apenas se nao for primeiro acesso. Apos sucesso, recarrega o usuario do /api/users/me/ e redireciona ao dashboard apropriado.

AdminDashboard.jsx

Tela inicial do admin. Chama /api/dashboard/overview/ e renderiza tres StatCards: Professores registrados, Disciplinas, Respostas.

Professores.jsx

CRUD completo de professores. Admin pode listar, cadastrar (com senha provisoria), editar (incluindo trocar a senha) e excluir. A senha definida aqui sempre vira com `must_change_password=True` do backend, forçando o professor a troca-la no primeiro login.

Disciplinas.jsx

CRUD de disciplinas. No cadastro, apos POST em /api/courses/, atualiza o google_form_link para apontar para a rota publica do frontend (/formulario/<id>) e o backend regenera o QR Code com a nova URL. As 24 perguntas padrao sao criadas automaticamente no save() do backend - o frontend nao precisa mais POSTar perguntas.

ProfessorDashboard.jsx

Tela inicial do professor. Lista as proprias disciplinas em uma barra lateral; ao selecionar, exhibe o QR Code (vindo do backend como imagem) ou permite gerar/baixar o relatorio PDF chamando /api/reports/course/<id>/pdf/.

Relatorios.jsx

Tela compartilhada entre admin e professor. Permite selecionar uma disciplina e ver o grafico de barras das medias por pergunta (usando Recharts), com botao para baixar o PDF completo.

EvaluationForm.jsx

Tela publica acessivel via QR Code. Rota /formulario/<courseId>. Carrega as perguntas via GET /api/evaluations/questions/?course_id=X (sem token), exhibe cada pergunta com radio buttons (0 a 5 + "Nao sei avaliar") ou textarea para comentarios. Ao enviar, POSTa em /api/evaluations/submit/ tambem sem token. Exhibe tela de agradecimento ao terminar.

5.4 App.jsx - Rotas e protecao

Define todas as rotas da SPA:

/	Login (publica)
/trocar-senha	SetPassword (logado)
/admin	AdminDashboard (logado, nao em troca de senha)
/professores	Professores (idem)
/disciplinas	Disciplinas (idem)
/relatorios	Relatorios (idem)
/professor	ProfessorDashboard (idem)
/formulario/:courseId	EvaluationForm (publica)

O componente *PrivateRoute* protege as rotas autenticadas: se nao houver token, redireciona para /; se o usuario tem must_change_password=true, redireciona forcadamente para /trocar-senha (impedindo "pular" o fluxo de primeiro login).

6. Modelo de Dados (Banco)

O esquema relacional do PostgreSQL tem cinco tabelas principais. A integridade é mantida por ForeignKeys com ON DELETE CASCADE - ao apagar um Course, todas as perguntas e respostas vinculadas também desaparecem.

users_user (User)

id	institutional_email	role	password	first_name	must_change_password
1	admin@ft.unicamp	admin	hash...	Admin	False
2	ana@ft.unicamp	professor	hash...	Ana	True

courses_course (Course)

id	name	code	professor_id	google_form_id	qr_code_img
1	Estatística	EST001	2	.../form/1	qr_..._1.png

evaluations_evaluationquestion

id	course_id	order	type	text
1	1	1	scale	A infraestrutura da sala...
24	1	24	text	Indique sugestões...

evaluations_evaluationresponse (sem FK para usuario! anonimo)

id	course_id	submitted_at
1	1	2026-05-03

evaluations_evaluationanswer

id	response_id	question_id	scale_value	text_value
1	1	1	4	NULL
2	1	23	NULL	Otima!

Anonimato: é fisicamente impossível descobrir quem deu uma resposta. EvaluationResponse não guarda IP, sessão, nem qualquer vínculo com User. O QR Code apenas direciona o navegador para o formulário; não identifica o aluno.

7. Fluxos de Funcionamento

7.1 Cadastro de professor pelo admin

[Admin abre tela /professores]

- > clica em "Cadastrar professor"
- > preenche nome, sobrenome, e-mail institucional, senha provisoria
- > clica em "Cadastrar"

Frontend: POST /api/users/professores/

Authorization: Bearer <admin token>

body: { first_name, last_name, institutional_email,
role: 'professor', password: 'provisoria123' }

Backend: UserCreateSerializer.create()

- cria User com role=professor
- aplica password via set_password (hash)
- FORÇA must_change_password = True

Resposta: 201 Created

Lista de professores recarrega na tela.

7.2 Primeiro login do professor (troca obrigatoria de senha)

[Professor abre /]

- > digita e-mail e senha provisoria
- > clica em Entrar

Frontend: POST /api/auth/login/ -> recebe access token JWT

Frontend: GET /api/users/me/ -> { must_change_password: true, ... }

Frontend: redireciona para /trocar-senha

[Tela SetPassword]

- > NAO pede senha atual (e o primeiro login)
- > professor digita nova senha + confirmacao
- > clica em Salvar

Frontend: POST /api/users/me/change-password/

body: { new_password: 'NovaSegura123' }

Backend: ChangePasswordView.post()

- como must_change_password=True, nao exige current_password
- aplica set_password
- zera must_change_password = False

Frontend: GET /api/users/me/ -> recarrega user no localStorage

Frontend: redireciona para /professor (dashboard normal)

7.3 Cadastro de disciplina e geracao de QR Code

[Admin em /disciplinas]

- > clica em "Cadastrar disciplina"
- > preenche nome, codigo, professor responsavel, matriculados
- > clica em "Cadastrar"

Frontend: POST /api/courses/

body: { name, code, professor: <id>, enrollment_count,
google_form_link: 'http://localhost:5173/formulario/0' }

Backend: Course.save()

- 1) detecta criacao (self._state.adding=True)
- 2) gera QR Code do google_form_link via qrcode.make()
- 3) salva PNG em media/qrcodes/qr_course_<code>.png

- 4) `super().save()` persiste a Course no banco (recebe id)
- 5) chama `sync_standard_questions(self)`
 - > cria 24 `EvaluationQuestion` (orders 1 a 24)

Resposta: 201 Created { id: 5, ... }

Frontend: PATCH `/api/courses/5/`
body: { google_form_link: 'http://localhost:5173/formulario/5' }

Backend: `Course.save()` detecta mudanca de link
-> regenera QR Code apontando para a URL correta

Disciplina pronta: aparece na lista, com QR Code clicavel e ja com as 24 perguntas oficiais associadas.

7.4 Aluno responde o formulario via QR

[Em sala, professor projeta o QR Code]
-> aluno aponta camera do celular
-> celular abre `http://localhost:5173/formulario/5`

Frontend (`EvaluationForm.jsx`):

- rota PUBLICA (nao exige login)
- GET `/api/evaluations/questions/?course_id=5` (AllowAnonymous)
- renderiza as 24 perguntas em ordem:
 - * scale: radio buttons 0..5 (Nao sei | DT | DP | Neutro | CP | CT)
 - * text: textarea para comentario

[Aluno preenche e clica Enviar]

Frontend: POST `/api/evaluations/submit/` (AllowAnonymous)
body: {
 course: 5,
 answers: [
 { question: 1, scale_value: 4, text_value: '' },
 { question: 2, scale_value: 5, text_value: '' },
 ...
 { question: 23, scale_value: null, text_value: 'Otima aula' },
 { question: 24, scale_value: null, text_value: 'Mais exemplos' }
]
}

Backend: `EvaluationResponseSerializer.create()`

- cria 1 `EvaluationResponse` (anonima, sem user_id)
- cria N `EvaluationAnswer` ligadas a essa Response

Frontend: mostra tela de agradecimento.

7.5 Professor consulta dashboard e relatorios

[Professor logado em `/professor`]
-> ve lista das proprias disciplinas (filtradas no backend)
-> clica em uma disciplina -> ve QR Code
-> clica em Relatorio

Frontend: GET `/api/reports/course/5/pdf/` (Bearer token)
responseType: 'blob'

Backend: `CoursePdfReportView.get()`

- confere que essa Course pertence ao professor logado
- chama `generate_course_pdf_report(course)`
- retorna `FileResponse` do PDF

Frontend: cria Blob -> abre download no navegador

7.6 Geracao interna do relatorio PDF

`generate_course_pdf_report(course):`

1. Calcular respondentes, matriculados, razao, tamanho amostral
2. Montar `header_block` com tabela institucional
3. Para cada question ordenada por `order`:
 - Se `scale`:
 - `get_scale_values()`
 - `calculate_mean_sd()`
 - `create_histogram()` via `matplotlib` (PNG temporario)
 - inserir no PDF com legenda DT/DP/...
 - Se `text`:
 - listar todos os comentarios numerados
- Sempre que muda de secao (A->B->C->D),
 - imprimir titulo da secao
4. `doc.build()` com footer custom em todas as paginas

8. API REST - Endpoints Disponiveis

Todas as rotas estao sob o prefixo **/api/**. O cabeçalho *Authorization: Bearer <token>* e obrigatorio em todas, exceto onde indicado como publica.

Autenticacao

Metodo	Rota	Permissao	Descricao
POST	/api/auth/login/	publica	Recebe e-mail + senha, devolve token JWT.
POST	/api/auth/refresh/	publica	Renova o token de acesso.

Usuarios

Metodo	Rota	Permissao	Descricao
GET	/api/users/me/	logado	Dados do usuario atual + flags.
POST	/api/users/me/change-password/	logado	Define/troca a propria senha.
GET	/api/users/professores/	admin	Lista professores.
POST	/api/users/professores/	admin	Cria um professor novo.
GET	/api/users/professores/<id>/	admin	Detalhe de um professor.
PATCH	/api/users/professores/<id>/	admin	Edita um professor.
DELETE	/api/users/professores/<id>/	admin	Exclui um professor.

Disciplinas

Metodo	Rota	Permissao	Descricao
GET	/api/courses/	logado	Lista (admin: todas; prof: as suas).
POST	/api/courses/	admin	Cria disciplina (gera QR + 24 perguntas).
GET	/api/courses/<id>/	logado	Detalhe da disciplina.
PATCH	/api/courses/<id>/	admin	Edita disciplina.
DELETE	/api/courses/<id>/	admin	Exclui disciplina.

Avaliaco

Metodo	Rota	Permissao	Descricao
GET	/api/evaluations/questions/?course_id=X	publica	Perguntas do formulario.
POST	/api/evaluations/questions/	admin	Cria pergunta avulsa.
POST	/api/evaluations/submit/	publica	Aluno submete respostas.
GET	/api/evaluations/report/<id>/	prof/admin	Dados agregados p/ relatorio.

Dashboard e Relatorios

Metodo	Rota	Permissao	Descricao
GET	/api/dashboard/overview/	admin	Totais (professores, disciplinas, respostas).
GET	/api/dashboard/course/<id>/	prof/admin	Medias por pergunta.
GET	/api/reports/course/<id>/pdf/	prof/admin	Download do PDF do relatorio.

9. Seguranca, Anonimato e Privacidade

Camadas de protecao

- bullet **Autenticacao por JWT:** token de curta duracao (4h) carregado no header de toda requisicao autenticada. Sem token valido, o backend responde 401.
- bullet **Senhas hasheadas:** armazenadas com PBKDF2 (algoritmo default do Django). O banco nunca tem senha em texto claro.
- bullet **Validadores de senha:** AUTH_PASSWORD_VALIDATORS configurados impedem senhas curtas, comuns, totalmente numericas ou parecidas com atributos do usuario.
- bullet **Permissoes por papel:** classes IsAdminRole e IsProfessorOrAdmin isolam o que admin e professor podem fazer no nivel da view.
- bullet **Filtro de queryset:** alem da permissao, o backend filtra os querysets para que professor so receba dados das proprias disciplinas, mesmo se ele tentasse iterar IDs na URL.
- bullet **Primeiro login forçado:** o flag `must_change_password` garante que o professor nunca opere o sistema com a senha provisoria definida pelo admin.

Anonimato dos alunos

O modelo `EvaluationResponse` **nao tem ForeignKey para User**, nao registra IP nem identificador de sessao. O formulario publico (`/api/evaluations/submit/`) e `AllowAny` - nao exige login. Resultado: e impossivel correlacionar uma resposta a um aluno especifico, mesmo com acesso direto ao banco.

Pontos a reforcar antes de producao

- bullet **CORS_ALLOW_ALL_ORIGINS** deve ser substituido por uma whitelist do dominio do frontend.
- bullet **DEBUG = False** obrigatorio em producao (vaza tracebacks).
- bullet **ALLOWED_HOSTS** com o dominio real (nao `"*"`).
- bullet **SECRET_KEY** deve vir de variavel de ambiente, nunca commitada.
- bullet **HTTPS** obrigatorio - JWT em HTTP claro pode ser interceptado.
- bullet **Rate limit no /submit/** seria prudente para impedir spam de respostas no formulario publico.

10. Proximos Passos

Funcionalidades

- bullet Fluxo de recuperacao de senha por e-mail ("Esqueceu sua senha?" hoje e apenas visual no Login).
- bullet Painel de administracao para edicao/remocao das perguntas padrao diretamente pela interface, alem da lista hardcoded em Python.
- bullet Tela de detalhe da resposta - hoje so existe agregado por pergunta.
- bullet Exportacao em CSV para professor analisar dados brutos.
- bullet Tela com historico das avaliacoes anteriores por disciplina (por semestre).
- bullet Internacionalizacao (i18n) - hoje todos os textos estao em portugueses.

Hospedagem

- bullet Subir o codigo no GitHub (dois repositorios: backend e frontend).
- bullet Backend e PostgreSQL no Render.com (tier gratuito); frontend no Vercel ou Render Static.
- bullet Ajustes obrigatorios antes do deploy: dj-database-url, gunicorn, whitenoise, variaveis de ambiente, VITE_API_URL no frontend.
- bullet Para uso real continuado pela FT, considerar servidor proprio da UNICAMP (verificar com o orientador).

Documentacao da Monografia (TCC)

Este documento cobre a parte tecnica. Para a monografia do TCC ainda faltam: fundamentacao teorica (avaliacao institucional, tecnologias web, LGPD), metodologia, capitulo de resultados (printscreens + metricas), testes e validacao, conclusao, trabalhos futuros e referencias bibliograficas em ABNT.

Fim do documento.